Academic Project Planning and Automated Architecture Design Tool

Software Requirements Specification [SRS]

Version: 1.1

Name: - Akhya Sinha **RegNo.:-** 23BKT0045

Signatures

Date	Revision	Approved By

List of Contributors

Name	Initials	Organization	Email

Change History

Revision	Date	Description
1.0	2026.01.23	Initial Revision
1.1		Second Revision
1.2		First Final Revision

Preface

This document presents the Software Requirements and design overview for the Academic Project Planning and Automated Architecture Design Tool, a system developed to assist students in effectively planning and designing academic software projects. In many educational environments, students encounter challenges during the initial phases of software development, particularly in project planning and selecting an appropriate system architecture. These challenges often lead to inefficient time management, unclear design structures, and inconsistent project outcomes.

The proposed tool aims to address these issues by providing an automated, rule-based solution that generates a phase-wise project plan and recommends a suitable high-level software architecture based on basic project parameters. The system emphasizes simplicity, clarity, and ease of use, making it well suited for academic and laboratory settings. By automating early-stage decision-making, the tool helps students better understand software engineering concepts such as project planning, system design, and architectural structuring.

This document outlines the purpose, scope, functional requirements, and design considerations of the system, serving as a comprehensive reference for development, evaluation, and future enhancement of the project while aligning with standard software engineering documentation practices.

Table of Contents

1 Introduction i
1.1 Purpose i
1.2 Scope i
1.3 Definitions, Acronyms, and Abbreviations i
1.4 References i
2 Overall Description ii
2.1 Product Perspective ii
2.2 Product Functions ii
2.3 Constraints iii
2.4 User Characteristics iv
2.5 Entity Relationships v
2.6 Data Flows vi
2.7 Data Dictionary viii
2.8 State Transitions xixi
2.9 Future Scope of Development xi
2.9.1 Advanced Planning techniques. Xi
2.9.2 AI-based Architecture Recommendations xii
2.9.3 Agile Tool Integration xii
2.9.4 Report Generation xii
2.9.5 Multi – User support xiii
2.9.6 Graphical Architecture Visualization xiv
2.9.7 Customization of Rules xiv
2.9.8 Cloud Based Deployment xiv
2.10 Assumptions xi
3 Business xi
3.1 Business Plan. Xi
3.2 Business Effect xii
4 Specific Requirements xii
4.1 System Features xii
4.1.1 Automated Project Planning xii
4.1.2 High-Level Architecture Recommendation xiii

Table of Contents

4.1.3 Rule-Based Decision Engine xiv
4.1.4 User-Friendly Interface xvixvi
4.1.5 Input Validation Mechanism..... xvii
4.2 Performance Requirements xviii
4.3 Design Constraints xviiixviii
4.4 Standards Compliance.....xix
4.5 Software System Attributes.....xixxix
4.5.1 Reliability xix
4.5.2 Security xx
4.5.3 Maintainability xx
4.5.4 Usability.. xx
4.5.5 Interoperability..... xxi
4.5.6 Compliance and Regulatory Requirements.. xxi
4.5.7 Disaster Recovery and Backup Objectives xxi
4.5.8 Portability xxi
Appendix A – State Transition Diagrams xxiii

List of Figures

Figure 1 Entity Relationship Diagram v
Figure 2 Data Flow Diagram vi

List of Tables

Table of Definitions, Acronyms, and Abbreviations i
Table of References i
Table of Shall Requirements ii
Table of Design Constraints iii
Table of User Characteristics iv
Table of Data Dictionary viii
Table of Performance Requirements xviii
Table of Design Constraints xviii
Table of Standards Compliance xix

1 Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) document is to clearly define and describe the functional and non-functional requirements of the Academic Project Planning and Automated Architecture Design Tool. This document serves as a formal reference for understanding system behavior, constraints, and design considerations before development begins. The project aims to assist students in planning academic software projects by automatically generating a phase-wise project schedule and recommending a suitable high-level software architecture based on basic project inputs. By automating early-stage planning and design decisions, the system helps improve clarity, consistency, and adherence to software engineering principles in academic project development..

1.2 Scope

The scope of the Academic Project Planning and Automated Architecture Design Tool is limited to supporting early-stage planning and high-level design of academic software projects. The system focuses on collecting basic project parameters such as project type, team size, duration, and technology to automatically generate a phase-wise project plan and recommend a suitable software architecture. It is intended for use in software engineering laboratories, mini projects, and final-year academic projects. The project does not include detailed effort estimation, cost analysis, low-level design, or code generation. The tool is designed to be simple, rule-based, and easily extensible for future enhancements.

1.3 Definitions, Acronyms, and Abbreviations

Table of Definitions, Acronyms, and Abbreviations

Definition, Acronym, or Abbreviation	Description
SRS	Software Requirements Specification.
SDLC	Software Development Life Cycle
Monolithic Architecture	A software architecture where all components are developed and deployed as a single unit
Agile	A software development methodology that emphasizes iterative development, flexibility, and collaboration
API	Application Programming Interface
Rule-Based System	A system that makes decisions using predefined rules and logical conditions.

1.4 References

Table of References

References	Description
I. Sommerville, Software Engineering, 10th ed., Boston, MA, USA: Pearson Education, 2016	This reference provides fundamental concepts of software project planning, system design, and architectural principles used in the project.
K. Wiegers and J. Beatty, Software Requirements, 3rd ed., Redmond, WA, USA: Microsoft Press, 2013.	This book is used as a primary reference for understanding and preparing Software Requirements Specification (SRS) documents and requirement analysis.
P. Kruchten, "Architectural Blueprints—The '4+1' View Model of Software Architecture," IEEE Software, vol. 12, no. 6, pp. 42–50, Nov.–Dec. 1995	This paper provides foundational knowledge on high-level software architecture design and architectural decision-making

2 Overall Description

2.1 Product Perspective

The Academic Project Planning and Automated Architecture Design Tool is designed as a standalone, lightweight decision-support system intended for use in academic and laboratory environments. It is not a full-scale project management or architectural modeling platform; instead, it focuses on supporting the early stages of the software development life cycle. The tool operates independently and does not require integration with complex enterprise systems or external services. It accepts basic project parameters from users and applies predefined, rule-based logic to generate a phase-wise project plan and recommend suitable high-level software architecture. The system follows a layered architecture, separating the user interface, business logic, and decision engine to ensure modularity and maintainability. This project complements existing software engineering practices by providing structured guidance for students, helping them better understand project planning and architectural design concepts while remaining simple, transparent, and easy to extend for future academic enhancements.

2.2 Product Functions

The follow is a table of the requirements that the system SHALL meet. The list of requirements was produced from the initial project documentation provided by the requirements expert.

Table of Shall Requirements

ID	Origin	Shall Requirement
01	*Project Scope Definition*	The system SHALL allow the user to enter basic project details such as project type, team size, duration, and technology.
02	*Project Planning Requirement*	The system SHALL automatically generate a phase-wise project plan based on the provided project inputs.
03	*Architecture Design Requirement*	The system SHALL suggest a suitable high-level software architecture using predefined decision rules.

04	*Automation Requirement*	The system SHALL perform project planning and architecture recommendation without manual calculations.
05	*User Interaction Requirement*	The system SHALL provide a user-friendly interface to collect inputs and display outputs.
06	*Output Presentation Requirement*	The system SHALL display the generated project plan in a clear and structured format.
07	*Design Support Requirement*	The system SHALL assist students in early-stage software design decisions for academic projects.
08	*Academic Usage Requirement*	The system SHALL support planning for both mini projects and final year academic projects.

2.3 Constraints

The follow is a table of the design constraints that the system SHALL meet. The list of constraints was produced from the initial project documentation provided by the requirements expert.

Table of Design Constraints

ID	Origin	***Shall Requirement***
1	*Academic Environment*	*The system SHALL be designed specifically for academic and laboratory use only.*
2	*Technology Constraint*	*The system SHALL be implemented using Python and related open-source libraries.*
3	*Simplicity Requirement*	*The system SHALL use a rule-based approach instead of complex AI or machine learning models.*
4	*Time Constraint*	*The system SHALL generate outputs within a short response time suitable for live demonstrations.*
5	*Platform Constraint*	*The system SHALL run on standard desktop or laptop environments.*
6	*Deployment Constraint*	*The system SHALL be deployed locally and not require cloud infrastructure.*

ID	Origin	***Shall Requirement***
7	*User Skill Constraint*	*The system SHALL be usable by students with basic computer knowledge.*
8	*Interface Constraint*	*The system SHALL provide a simple console-based or basic web-based user interface.*
9	*Data Constraint*	*The system SHALL not require large datasets for operation.*
10	*Security Constraint*	*The system SHALL not store sensitive personal or financial data.*
11	*Integration Constraint*	*The system SHALL support conceptual integration with Agile tools such as GitHub or Jira without mandatory API usage.*
12	*Scalability Constraint*	*The system SHALL be optimized for small-scale academic projects only.*
13	*Accuracy Constraint*	*The system SHALL rely on predefined decision rules for consistent output generation.*
14	*Maintenance Constraint*	*The system SHALL allow easy modification of rules without major code changes.*
15	*Documentation Constraint*	*The system SHALL be documented using standard software engineering documentation practices.*
16	*Compliance Constraint*	*The system SHALL follow standard SDLC and SRS guidelines taught in software engineering courses.*
17	*Resource Constraint*	*The system SHALL operate efficiently without requiring high computational resources.*
18	*Extensibility Constraint*	*The system SHALL be designed to allow future enhancements without redesigning the entire system.*

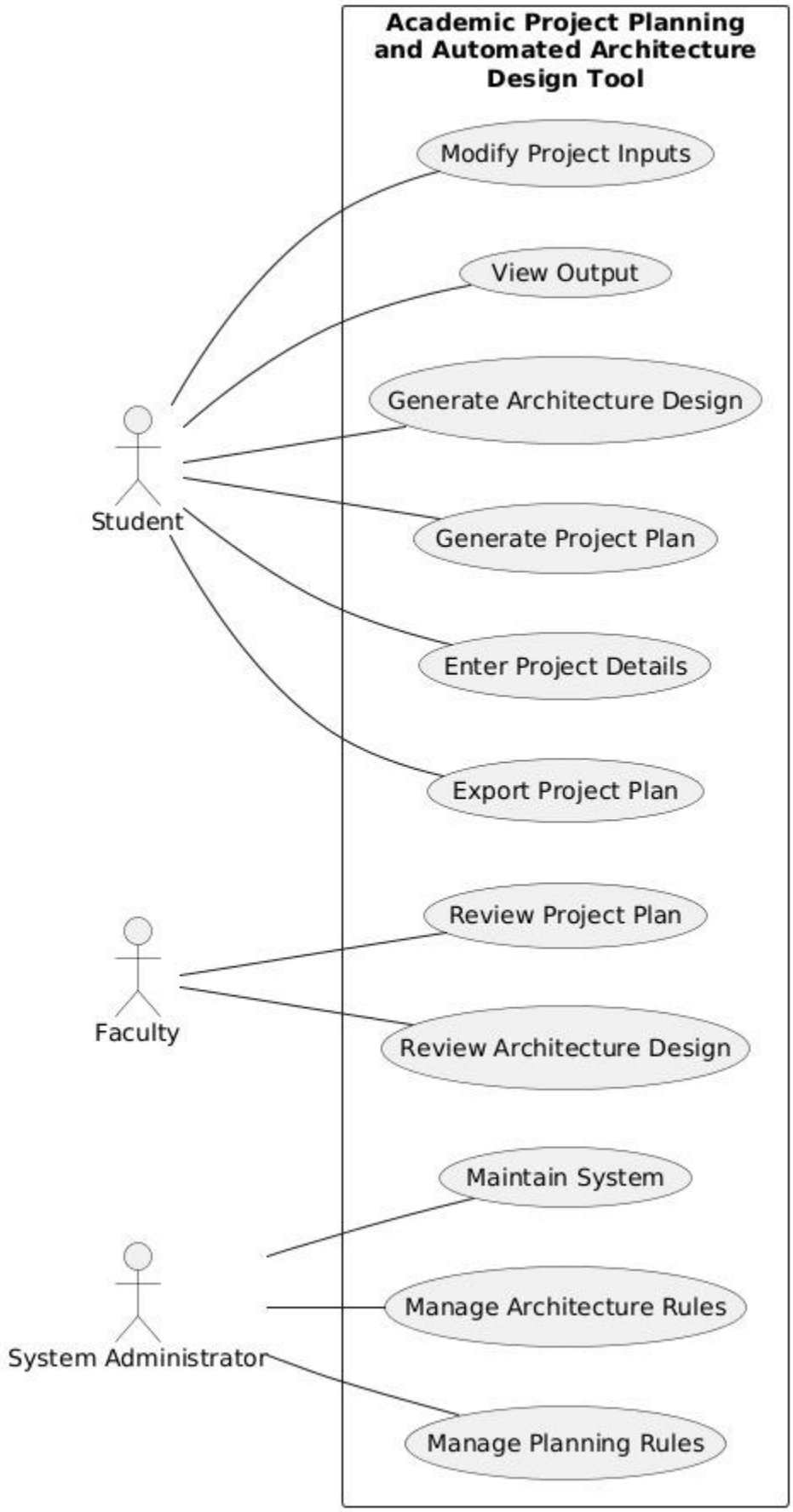
2.4 User Characteristics

The following table identifies and describes the different users of the Project Management Software. The information gathered about the different users of the system helped define what the software needs to do. Also, these users are referenced in the requirements and diagrams.

Table of User Characteristics

User	Description
Student	The student is the primary user of the system and uses the tool to plan academic software projects and obtain high-level architectural recommendations.

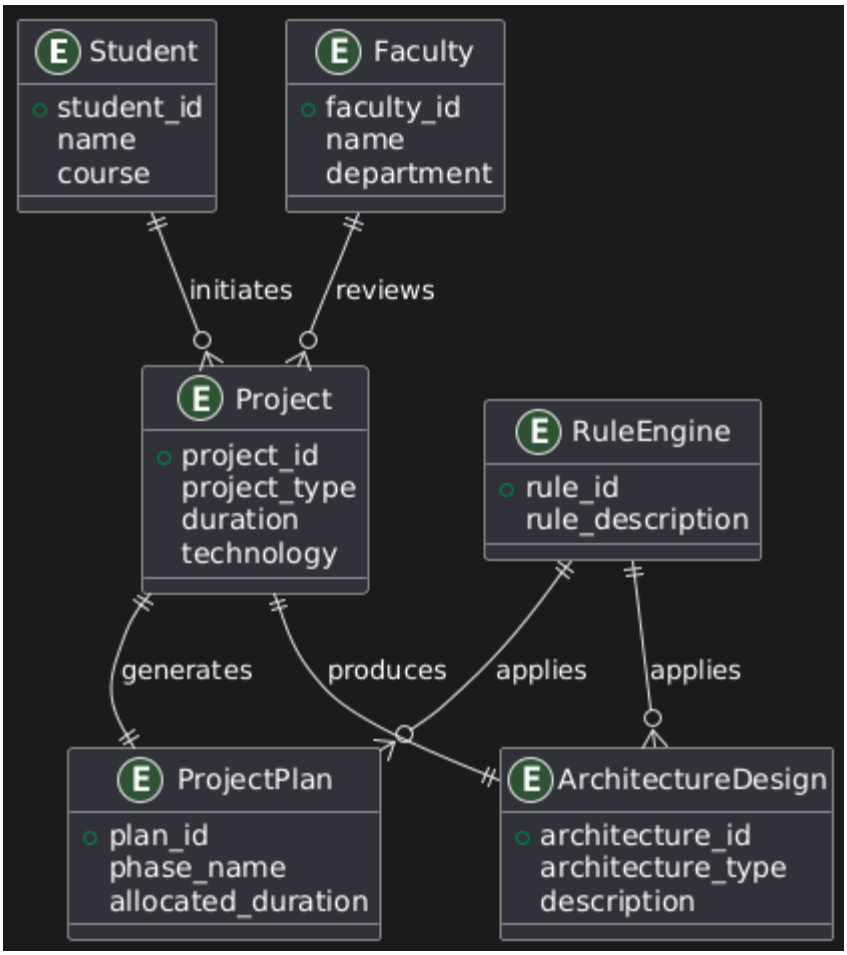
Faculty	The faculty member uses the system to review generated project plans and architecture designs for evaluation and guidance purposes.
Project Guide	The project guide assists students by reviewing the planning outputs and ensuring alignment with academic requirements.
System Administrator	The Administrator user will be computer literate and technically competent in performing administration on computer systems.
Lab Evaluator	The lab evaluator uses the system to assess the correctness and completeness of project planning and design outputs during evaluations.


Academic Project Planning and Automated Architecture Design Tool
Modify Project Inputs
View Output
Generate Architecture Design
Generate Project Plan
Enter Project Details
Export Project Plan
Review Project Plan
Review Architecture Design
Maintain System
Manage Architecture Rules
Manage Planning Rules
Student
Faculty
System Administrator

2.5 *Entity Relationships*

Figure 1 shows the entity relationships for the Academic Project Planning and Automated Architecture Design Tool.

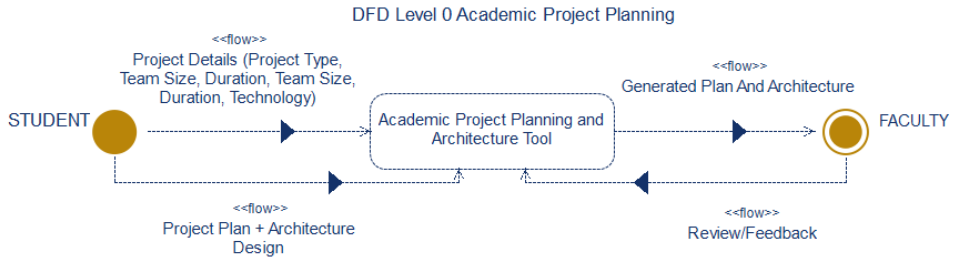
Figure 1 Entity Relationship Diagram

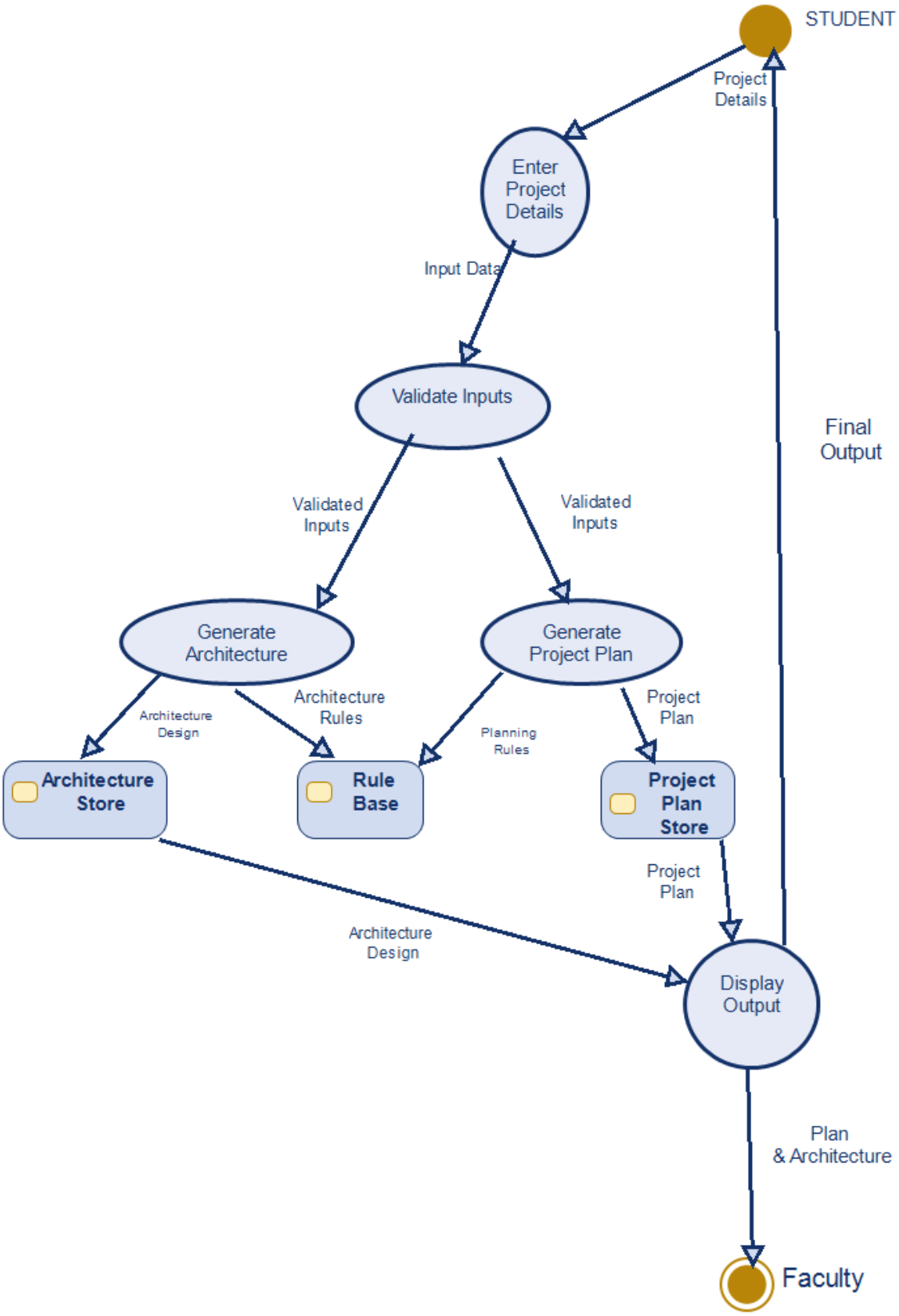


2.6 *Data Flows*

The following figures represent the data flow diagrams of the Academic Project Planning and Automated Architecture Design Tool. The first data flow diagram, figure 2, is the Context data flow. This is followed by the more detail data flow of the Software.

Figure 2 Data Flow Diagram





DFD Level 1 Architecture Planning Software

2.7 Data Dictionary

The follow tables in this section make up the data dictionary for the Academic Project Planning and Automated Architecture Design Tool. Using the Data Flow Diagrams, the following Data Dictionary elements were defined.

- Project Details
- Project Plan
- Architecture Design
- Rule Base
- User Inputs
- Generated Output
- Validation Data

Table of Data Dictionary

Data Dictionary Attribute	**Detail**
Name	Project Details
Aliases	Project Information
Where Used / How Used	Used as primary input for project planning and architecture recommendation
Description	Project Details represent the fundamental information required to analyze and plan an academic software project. This data includes the type of project (mini or final year), the number of team members involved, the total project duration in weeks, and the technology selected for development.
Supplementary Information	These details form the basis for all automated decisions made by the system and are validated before processing.

Data Dictionary Attribute	Detail
Name	Architecture Design
Aliases	High-Level System Architecture
Where Used / How Used	Generated as an output to guide system design decisions
Description	Architecture Design describes the recommended high-level structure of the software system, including the architectural style such as monolithic or three-tier architecture, and the interaction between major system components.
Supplementary Information	This data assists students in selecting an appropriate design approach early in the project lifecycle.

Data Dictionary Attribute	Detail
Name	Rule Base
Aliases	Decision Logic
Where Used / How Used	Used internally by the system for automated decision making
Description	The Rule Base contains a set of predefined logical conditions and decision rules that map user inputs to suitable project plans and architectural recommendations.
Supplementary Information	These rules ensure consistency, predictability, and transparency in system outputs.

Data Dictionary Attribute	Detail
Name	User Inputs
Aliases	Input Parameters
Where Used / How Used	Entered by the user through the system interface
Description	User Inputs consist of all information provided by the user, including project type, team size, project duration, and selected technology, which are required for system processing.
Supplementary Information	Input validation is performed to ensure correctness and completeness before further processing.

Data Dictionary Attribute	Detail
Name	Validation Data
Aliases	Input Verification Information
Where Used / How Used	Used during the input validation stage
Description	Validation Data stores the status and results of input verification checks, indicating whether the user-provided data is complete, valid, and suitable for processing.
Supplementary Information	This data prevents invalid or inconsistent inputs from affecting system outputs.

Data Dictionary Attribute	Detail
Name	Generated Output
Aliases	System Output
Where Used / How Used	Displayed to students and faculty after processing
Description	Generated Output represents the final results produced by the system, including the automated project plan and the recommended high-level architecture.
Supplementary Information	The output provides structured guidance for academic project execution and evaluation.

2.8 *State Transitions*

Please refer to Appendix A for the State Transition Diagrams. The first state transition diagram shows the state transitions. The second diagram in Appendix A shows the state transitions.

2.9 *Future Scope of Development*

2.9.1 *Advanced Planning Techniques*

Future versions of the system can integrate well-known software effort estimation models such as COCOMO, Function Point Analysis, or Use Case Point methods to improve the accuracy of project planning. Instead of equally dividing time across phases, the system would calculate effort based on project size, complexity, and development environment. This enhancement would help students understand realistic project estimation practices used in industry. It would also allow comparison between simple rule-based planning and mathematically derived estimates, thereby improving learning outcomes and making the tool more suitable for advanced academic and research-oriented projects.

2.9.2 *AI-Based Architecture Recommendation*

The current rule-based architecture recommendation can be enhanced using artificial intelligence techniques such as machine learning or knowledge-based systems. By analyzing historical project data, system characteristics, and outcomes, the tool could learn patterns and suggest more optimized architectural styles. This enhancement would allow adaptive and context-aware recommendations instead of fixed rules. Such an approach would expose students to modern AI-assisted software engineering practices and significantly increase the

system's intelligence, making it more suitable for complex projects and future industrial applications.

2.9.3 *Agile Tool Integration*

In future releases, the system can be integrated directly with Agile project management tools such as GitHub or Jira using their APIs. The generated project plan could automatically create GitHub issues, milestones, or Jira epics and sprints. This integration would bridge the gap between academic planning and real-world Agile execution. Students would gain hands-on exposure to industry-standard tools while following a structured plan generated by the system. This enhancement increases the project's practical relevance and supports its potential use in professional training environments.

2.9.4 *Report Generation*

The system can be enhanced to automatically generate detailed project planning and architecture reports in standard formats such as PDF or DOC. These reports could include project details, phase-wise plans, architecture descriptions, and diagrams. Such functionality would reduce manual documentation effort for students and ensure consistency in academic submissions. It would also help faculty in evaluation by providing standardized reports. This enhancement makes the tool more comprehensive by combining planning, design, and documentation support into a single system.

2.9.5 *Multi-User Support*

Future versions of the system can support multiple users with role-based access control. Different roles such as students, faculty, project guides, and administrators can be defined, each with specific permissions. Students could create and manage their projects, while faculty could review and provide feedback. Administrators could manage rules and system configurations. This enhancement would allow collaborative usage, improve scalability, and make the system suitable for institutional deployment across multiple courses or departments.

2.9.6 *Graphical Architecture Visualization*

Currently, the architecture recommendation is presented in a textual format. In the future, the system can automatically generate graphical architecture diagrams using UML or other visualization standards. These diagrams would visually represent system components and their interactions, making architectural concepts easier to understand. Graphical visualization would enhance learning, improve clarity, and make the tool more engaging for students. It would also align the system with standard software design documentation practices followed in academia and industry.

2.9.7 *Customization of Rules*

The system can be enhanced to allow users or administrators to customize planning and architecture rules without modifying the source code. A configuration-based or rule-editor interface could be provided to define new rules or modify existing ones. This would allow the tool to adapt to different academic curricula, institutional guidelines, or project types. Such flexibility improves maintainability and usability while encouraging experimentation

and deeper understanding of decision-making logic in software engineering.

2.9.8 *Cloud-Based Deployment*

The tool can be deployed as a cloud-based application to support remote access and collaborative project planning. Cloud deployment would allow students and faculty to access the system from anywhere without local installation. It would also enable data persistence, collaboration across teams, and centralized management. This enhancement would significantly increase scalability and adoption potential, transforming the tool from a local academic utility into a shared educational platform suitable for universities and training institutions.

2.10 *Assumptions*

The following table lists the assumptions made by the requirements that define software.

Assumption	**Description**
Secure Channel from Sender	The defined requirements assume that there is a method for secure transaction between the Sender and the

3 Business

3.1 Business Plan

Project Aspect	Description
Idea	Develop an EdTech SaaS tool that automates academic project planning and high-level software architecture design.
Problem Addressed	Students struggle with project planning and architecture selection due to lack of experience and guidance.
Solution Offered	A rule-based, automated tool that generates project plans and architecture recommendations instantly.
Target Market	Engineering colleges, universities, training institutes, and individual students.
Unique Selling Proposition (USP)	Simple, academic-focused automation unlike complex industry tools such as Jira or enterprise PM software.
Revenue Model	Freemium model with free academic version and paid premium features (reports, integrations, analytics).
Go-To-Market Strategy	Institutional adoption through colleges, lab integration, workshops, and student subscriptions.
Scalability Potential	Can scale into AI-based planning, cloud deployment, and multi-institution usage.
Competitive Advantage	Low learning curve, academic alignment, low cost, and fast deployment.

3.2 Business Effect

Impact Area	Effects
Educational Impact	Improves understanding of software engineering planning and architecture concepts among students.
Productivity Impact	Reduces time spent on manual planning and increases focus on implementation and learning.
Institutional Impact	Enables standardized project planning and evaluation across departments.
Economic Impact	Creates opportunities for affordable EdTech solutions and student-centric startups.
Skill Development	Enhances analytical thinking, planning

	skills, and architectural decision-making abilities.
Market Impact	Addresses a niche gap between academic needs and complex industrial tools.
Innovation Impact	Encourages adoption of automation and decision-support tools in education.

4 Specific Requirements

4.1 System Features

4.1.1 Automated Project Planning

4.1.1.1 Introduction

This feature enables the system to automatically generate a structured, phase-wise project plan for academic software projects based on basic project inputs provided by the user.

4.1.1.2 Functional Requirements

- *The system shall accept project details from the user.*
- *The system shall determine appropriate development phases.*
- *The system shall allocate project duration across phases automatically.*
- *The system shall generate and display the project plan.*

4.1.1.3 Stimulus Response

User Actions	System Actions
(1) Enters project type, duration, and team size	Reads and stores input data
(2) Submits project details	Validates input values
(3) Requests project plan	Calculates phase-wise duration
	Displays generated project plan

4.1.2 High-Level Architecture Recommendation

4.1.2.1 Introduction

This feature allows the system to recommend a suitable high-level software architecture based on project characteristics such as team size and technology.

4.1.2.2 Functional Requirements

- The system shall analyze project inputs.
- The system shall apply predefined architecture rules.
- The system shall recommend a suitable architecture.
- The system shall display the architecture recommendation.

4.1.2.3 Stimulus Response

User Actions	**System Actions**
Selects technology and team size	Reads project parameters

Requests architecture design	Applies architecture rules
	Determines suitable architecture
	Displays architecture recommendation

4.1.3 Rule-Based Decision Engine

4.1.3.1 Introduction

This feature provides the core logic of the system by applying predefined rules to automate project planning and architectural decision-making.

4.1.3.2 Functional Requirements

- *The system shall store predefined decision rules.*
- *The system shall apply rules consistently for the same inputs.*
- *The system shall support easy modification of rules.*

4.1.3.3 Stimulus Response

User Actions	System Actions
Provides project inputs	Sends inputs to rule engine
	Evaluates rules using conditional logic
	Returns planning and architecture decisions

4.1.4 User-Friendly Interface

4.1.4.1 Introduction

This feature ensures that all user inputs are complete, valid, and suitable for processing before generating outputs.

4.1.4.2 Functional Requirements

- *The system shall provide a clear input interface.*
- *The system shall display outputs in readable format.*
- *The system shall support basic navigation.*

4.1.4.3 Stimulus Response

User Actions	System Actions
Opens application	Displays input form
Enters project details	Accepts user inputs
Views results.	Displays outputs clearly

4.1.5 Input Validation Mechanism

4.1.5.1 Introduction

This feature ensures that all user inputs are complete, valid, and suitable for processing before generating outputs.

4.1.5.2 Functional Requirements

- *The system shall validate all user inputs.*
- *The system shall detect missing or invalid values.*
- *The system shall prevent processing of incorrect inputs.*

4.1.5.3 Stimulus Response

User Actions	System Actions
Enters project details	Checks input completeness
Submits invalid data	Displays validation error
Submits valid data	Allows processing to continue

4.2 Performance Requirements

The following tables list the performance requirements of the software.

Table of Performance Requirements

Performance Requirement	**Description**
Response Time	The system shall generate the project plan and architecture recommendation within a short time after user input submission.
Input Processing Time	The system shall validate and process user inputs efficiently without noticeable delay.
Rule Evaluation Speed	The rule-based decision engine shall evaluate planning and architecture rules quickly and consistently.
UI Responsiveness	The user interface shall remain responsive during input, processing, and output display.
Resource Utilization	The system shall operate using minimal CPU and memory resources suitable for academic environments.

4.3 Design Constraints

The table below lists those design constraints.

Table of Design Constraints

Design Constraint	Description
Academic-Oriented Design	The system shall be designed specifically for academic and laboratory use.

Technology Constraint	The system shall be implemented using Python and open-source frameworks only.
Rule-Based Logic	The system shall use predefined rule-based logic instead of machine learning techniques.
Platform Independence and Simplicity of Interface	The system shall run on standard desktop or laptop systems without special hardware. The system shall provide a simple and easy-to-use interface suitable for students.

4.4 Standards Compliance

The following table lists the different standards that the Academic Project Planning and Automated Architecture Design Tool project is to be in compliance with.

Table of Standards Compliance

Standard	**Description**
UML Standard	The system design shall use Unified Modeling Language (UML) for diagrams such as use case, activity, and sequence diagrams.
SDLC Guidelines	The system shall follow standard Software Development Life Cycle practices taught in software engineering courses.
Agile Practices	The system shall conceptually align with Agile principles for iterative planning and project management.

4.5 Software System Attributes

4.5.1 Reliability

Reliability refers to the ability of the system to consistently perform its intended functions without failure. In this project, reliability is ensured through deterministic, rule-based logic that produces the same output for the same input every time. Since the system does not depend on external services or real-time data sources, the chances of unexpected failures are minimized. Input validation mechanisms further enhance reliability by

preventing incorrect or incomplete data from being processed. Thorough testing using multiple input combinations ensures that the system behaves correctly under normal academic usage, making it dependable for laboratory demonstrations and evaluations.

4.5.2 Security

Security in this project focuses on protecting the system from misuse rather than handling sensitive data. The system does not collect or store confidential information such as personal identities or financial data, which reduces security risks. Input validation helps prevent invalid or malicious input from affecting system behavior. Since the application is intended for local academic use and operates in a controlled environment, advanced security mechanisms are not required. However, the system design ensures that internal logic and rule definitions cannot be altered unintentionally by users, maintaining integrity of automated planning and architecture recommendations.

4.5.3 Maintainability

Maintainability describes how easily the system can be modified, corrected, or enhanced over time. The project is designed with modular components such as input handling, rule-based decision logic, and output generation, which simplifies maintenance. The use of clear function definitions and readable Python code makes it easy for future developers or students to understand and update the system. Changes to planning logic or architecture rules can be made without affecting other parts of the system. Proper documentation and adherence to coding standards further support long-term maintainability.

4.5.4 Usability

Usability ensures that the system is easy to learn and use by its intended users. The tool provides a simple and intuitive user interface suitable for students and faculty with basic technical knowledge. Clear input labels, straightforward navigation, and readable output formats enhance user interaction. Accessibility considerations such as readable font sizes and clear visual layout are followed. Although the current version targets a single language, the design supports future internationalization and localization by separating user interface text from system logic. Overall, the system aims to minimize user effort while maximizing clarity and efficiency.

4.5.5 Interoperability

Interoperability refers to the ability of the system to operate across different platforms with minimal changes. The system is implemented using Python, which is platform-independent and can run on various operating systems such as Windows, Linux, and macOS. The application does not depend on platform-specific hardware or software, ensuring easy deployment in different academic environments. Interoperability is achieved by designing the system in a modular manner, allowing conceptual integration with external tools such as

GitHub or Jira through standardized interfaces or APIs in future extensions.

4.5.6 Compliance and Regulatory Requirements

The project is designed primarily for academic use and does not handle sensitive personal or financial data. Therefore, strict regulatory standards such as GDPR, HIPAA, or PCI-DSS are not directly applicable. However, the system follows general data protection principles by minimizing data collection and avoiding storage of confidential information. Basic accessibility guidelines are considered to ensure usability for a wide range of users. The documentation and development process align with standard software engineering and IEEE SRS guidelines.

4.5.7 Disaster Recovery and Backup Objectives

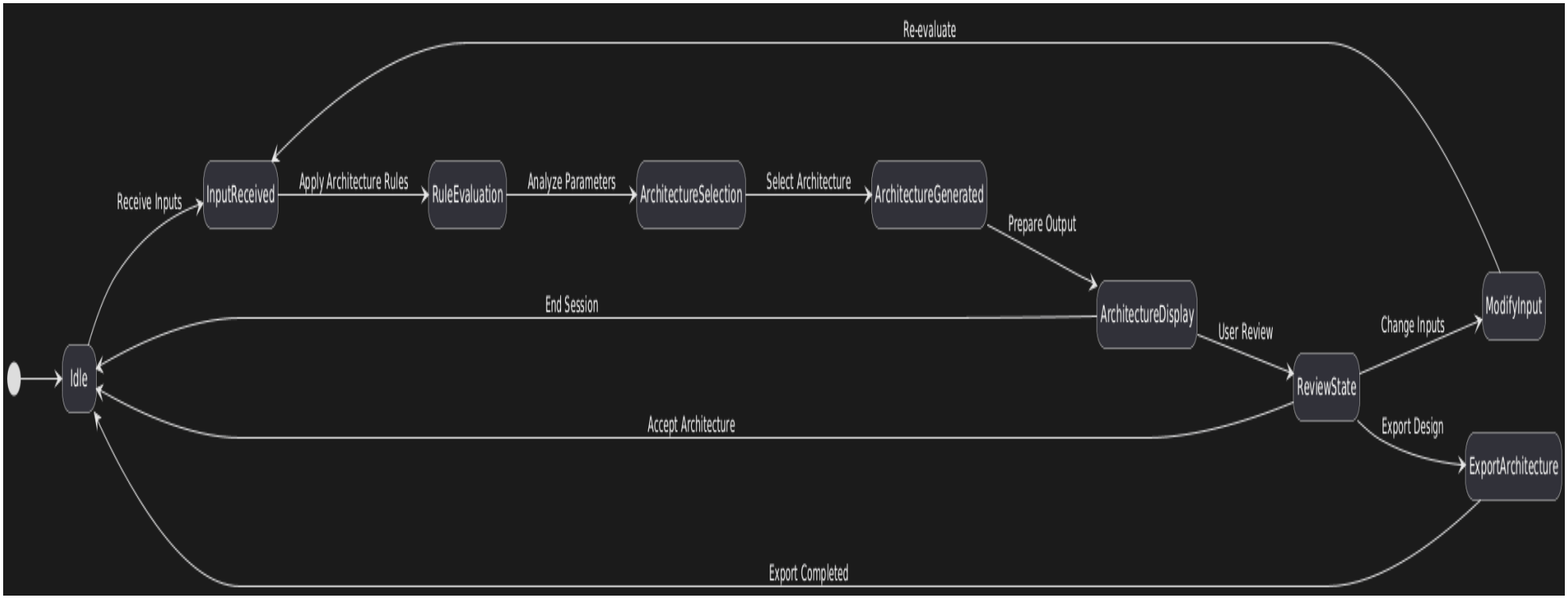
Disaster recovery ensures system availability in case of failures. Since the system is designed for local academic use, critical data is minimal and limited to project inputs and generated outputs. Basic backup mechanisms such as local file storage or database backups can be used to preserve data. The Recovery Time Objective (RTO) is minimal, as the system can be quickly restarted after failure. The Recovery Point Objective (RPO) is also low, as data can be regenerated by re-entering inputs if necessary. This approach ensures acceptable continuity for academic environments.

4.5.8 Portability

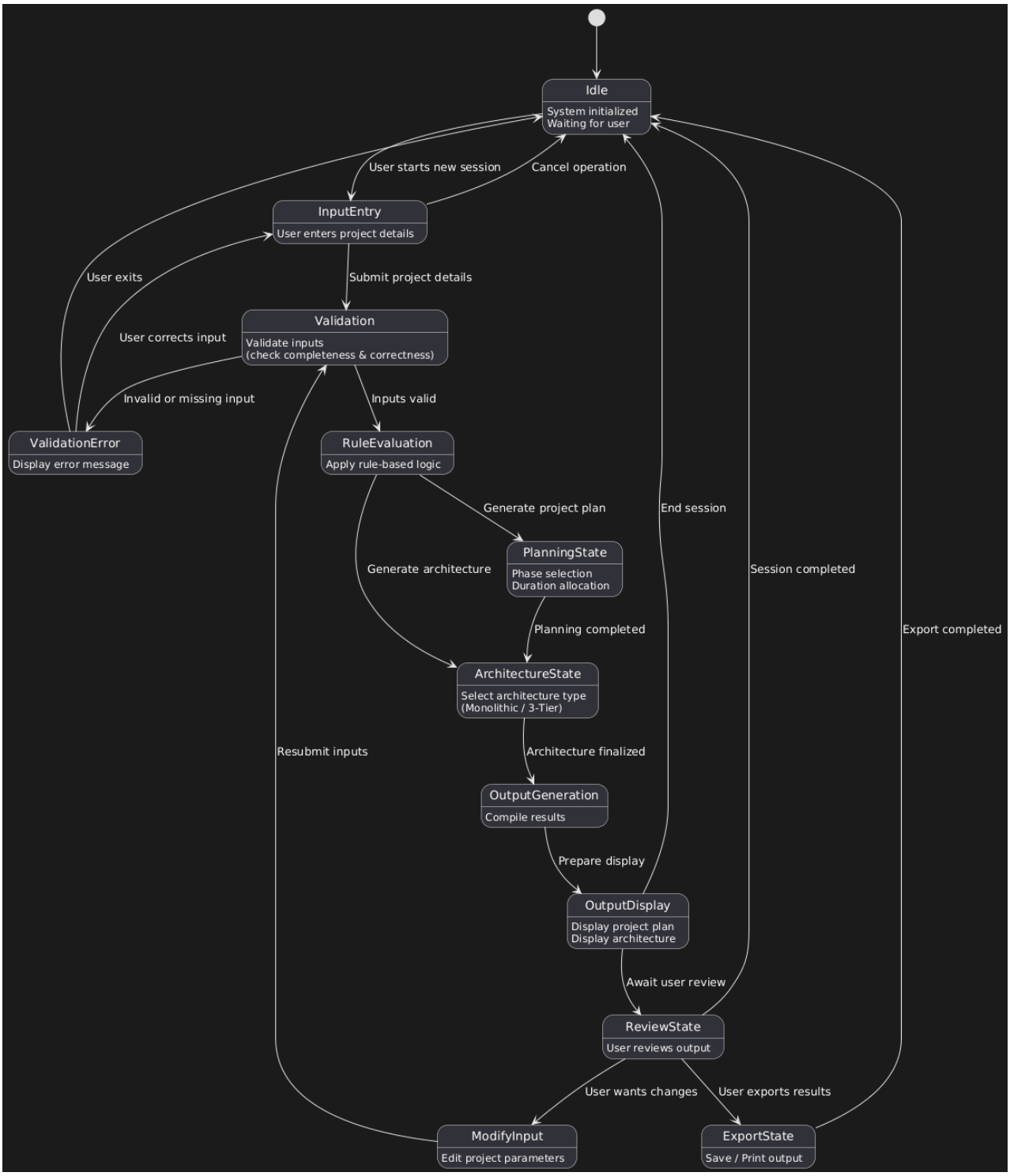
Portability refers to the ability of the system to operate across different platforms with minimal changes. The project is implemented using Python, a platform-independent programming language, which allows the system to run on various operating systems such as Windows, Linux, and macOS. The application does not rely on platform-specific features or hardware, ensuring smooth execution in different academic environments. Both console-based and web-based versions can be deployed locally without complex configuration. This portability makes the system suitable for use across multiple laboratories and student systems.

Appendix A – State Transition Diagrams

Planning State Transistion Diagram



Architecture State Transition diagram



Academic project Planning and Architecture Suggesting Tool.